I. THE FALL OF RNNs AND THE RISE OF TRANSFORMERS

In File 20, we studied RNNs and LSTMs. However, they have two critical bottlenecks that prevent them from scaling to billions of parameters:

1. **Sequential Processing:** RNNs read text one word at a time. They cannot process the 100th word until the 99th is finished, making it impossible to utilize modern GPU parallelism.
2. **Long-Range Dependency:** Even with LSTM gates, models struggle to connect information at the beginning of a long document to the end.

The Transformer (introduced in 2017) solved this by discarding recurrence entirely and replacing it with **Self-Attention**.

II. THE SELF-ATTENTION MECHANISM

Imagine reading the sentence: *"The animal didn't cross the street because **it** was too tired."* To understand what "**it**" refers to, a human brain automatically focuses on "**animal**."

Self-Attention allows a model to look at the entire sentence at once and calculate "weights" (importance) for every word relative to every other word in the sequence.

2.1. Queries, Keys, and Values (Q, K, V)

To calculate attention, every word is transformed into three distinct vectors:

- **Query (Q):** "What am I looking for?"
- **Key (K):** "What information do I contain?"
- **Value (V):** "What is my actual content?"

The model computes the similarity between a word's **Q** and all other words' **K**, then uses that score to create a weighted sum of the **V** vectors.

III. THE TRANSFORMER ARCHITECTURE

The standard Transformer consists of two main stacks:

1. **Encoder:** Reads the input text and builds a deep, contextual understanding (e.g., **BERT**).
2. **Decoder:** Predicts the next word in a sequence based on the context provided by the Encoder (e.g., **GPT**).

3.1. Multi-Head Attention

Instead of one "attention" process, Transformers use multiple "heads" working in parallel. One head might focus on grammar, another on entities, and a third on sentiment, allowing the model to understand language from multiple perspectives simultaneously.

IV. POSITIONAL ENCODING

Because Transformers process all words in parallel, they have no inherent sense of order (they don't know if "cat" comes before or after "dog"). To solve this, we add a **Positional Encoding** vector to each

word. This is a mathematical function (Sine/Cosine waves) that "stamps" each word with its position in the sentence.

V. WHY TRANSFORMERS ARE THE "HEART" OF YOUR RAG SYSTEM

Your RAG pipeline relies on this architecture for two reasons:

1. **Embeddings:** You use an Encoder (like BERT or E5) to convert your 50 PDF files into high-dimensional vectors.
2. **Contextual Generation:** When a user asks a question, the Transformer "attends" to the retrieved chunks from your library to generate an accurate, human-like response.

VI. PRACTICAL LAB: USING TRANSFORMERS WITH HUGGING FACE

In production, we use the transformers library to leverage pre-trained models.

Python

```
from transformers import pipeline


# 1. Initialize a Question-Answering pipeline

# This model uses the Transformer architecture to locate answers in text

qa_model = pipeline("question-answering", model="distilbert-base-cased-distilled-squad")


context = "Duy Tan University (DTU) is located in Da Nang, Vietnam. It has a high-priority AI program."

question = "Where is DTU located?"


# 2. The Transformer uses Attention to map the question to the context

result = qa_model(question=question, context=context)


print(f"Answer: {result['answer']} (Confidence: {result['score']:.4f})")
```